

CAPSTONE PROJECT

U.S. Stock Market Analysis & Modeling

Data Source: Stooq.com (U.S. Listed Stocks)

Scraped for educational and non-commercial research purposes

Exploratory Project Report

Project Overview

➤ Introduction

In this Capstone Project we Chose U.S Stocks data to perform a full analysis and created a model which included:

- 1. Web Scraping Script – Where we created a web scripting script
- 2. Data Cleaning & Preprocessing – Cleaned and preprocessed our data
- 3. Exploratory Data Analysis & Visualization – Performed Data Analysis and Visualization
- 4. Model Building & Evaluation – Built a model and evaluated the results
- 5. Interactive Dashboard – Created a PowerBI dashboard

We chose a dataset of U.S stocks that was scrapped from [www.stooq.com](https://stooq.com/t/?i=518&v=1) (<https://stooq.com/t/?i=518&v=1>).

This website is a Polish website that is public and does not require a login to access all or more pages on the site and unlike other major websites it does not contain a direct restriction for scrapping or non-commercial data gathering.

This capstone project focuses on the analysis and unsupervised clustering of U.S. listed stocks. The dataset covers approximately 4,000 after filtering unusable stocks listed on major U.S. exchanges and includes a wide range of price, volume, and return metrics collected across multiple timeframes.

The primary objective of this project is to perform extensive analysis from start and use machine learning algorithms like KMeans / LinearRegression within the stock based on trading behavior, price dynamics, and return patterns.

Note: This website is live and actively updates its values for all the columns for each stock daily on every day the market is open/live so each time we scrape the data will be different.

➤ Data Description

The data was collected from Stooq.com, a publicly accessible financial data platform. Stooq does not prohibit non-commercial scraping in its robots.txt or Terms of Service, and all pages used are accessible without login or authentication. The dataset is composed of three separate views scraped from the platform:

- **View 1** - Daily price data Columns like Open, High, Low, Last, Volume, Turnover, and daily Change
- **View 4** - Short to long-term return percentages Columns like 1D, 3D, 5D, 10D, 1M, 2M, 3M, 6M, and YTD returns, where D refers to days and M refers to Months
- **View 6** - 52-week metrics Columns 52-week price change percentage, 1-Year High

After scraping we had combined 21+2 feature Columns covering every major dimension of stock activity price, volume, short-term momentum, and long-term performance but here is the list of columns and their description:

Column	Description
Symbol	Unique stock ticker symbol.
Name	Company name.
Open	Opening price of the stock for the trading session.
High	Highest price reached during the trading session.
Low	Lowest price reached during the trading session.
Last	Closing/latest traded price.
Change	Percentage change in price from the previous trading day.
Volume	Total number of shares traded.
Turnover	Total value of shares traded.
Date	Trading date.
1D	1-day return (%) measuring price change over the last trading day.
3D	3-day return (%) measuring price change over the last 3 trading days.
5D	5-day return (%) measuring price change over the last 5 trading days.
10D	10-day return (%) measuring price change over the last 10 trading days.
1M	1-month return (%) measuring price performance over approximately 1 month.
2M	2-month return (%) measuring price performance over approximately 2 months.
3M	3-month return (%) measuring price performance over approximately 3 months.
6M	6-month return (%) measuring price performance over approximately 6 months.
YTD	Year-to-date return (%) from the beginning of the current year.
52W Change	Percentage price change over the last 52 weeks.
1Y High	Highest stock price recorded during the last 1 year.
1Y Low	It was Empty When we scraped the data

Out of all the 21+2 Columns 2 were empty and were removed during preprocessing. After merging all three files the final dataset contains approximately 6000 rows and 21 Usable Feature Columns.

Issue: One major issue we found was that the website updates regularly and few columns get disabled at the time of scraping all the columns present in the dataset were active and visible on the live website.

➤ Project Goals

Data Collection and Web Scraping

The first task was building a fully automated web scraper to collect data from any publicly accessible data platform. In My project www.Stooq.com was used for Scraping of U.S Financial stock data.To solve the problem of **Stock filtering and screening**. Stooq stores its data in a multi page views in a already tabular format with simple pagination, on each page it contains only 100 results. The scraper was built using Selenium to simulate a real browser session and BeautifulSoup to parse the rendered HTML output.

Data Cleaning & Preprocessing

The second task focused on transforming the Scrapped data into a analysis-ready dataset. This involved merging all the scrapped data, formatting columns as needed, handling any missing data and fixing data types ensuring all the data present in the dataset was cleaned and formatted in a manner that it can be used for training a machine learning model and perform **Descriptive Data Analysis and Visualizations**

Model Creation & Evaluation

In this task, stock market data will be prepared and analyzed using return, liquidity, volume, turnover, and performance-related features. The model will be chosen according to the data how it is distributed and spread out and if there is a presence of outliers or not and if the data allows clustering, regression or classification models to be built on it. The results will then be evaluated using statistical and visual methods to ensure the findings are reliable and Model performance is optimal.

Interactive Dashboard

An interactive dashboard was developed to present key insights from the stock market dataset in a clear and user-friendly manner. The dashboard enables users to explore stock performance, return trends across multiple time periods, and segment-wise characteristics through dynamic visualizations and filtering options. Key performance indicators (KPIs), trend charts, and comparative visuals help identify market patterns and support data-driven decision-making. Interactive features such as slicers, drill-through navigation and custom visuals enhance exploration and analysis of the data.

Web Scrapping

The data was collected using a custom-built Python scraper combining Selenium and BeautifulSoup to extract stock data from Stooq.com across three separate URL views. A detailed walkthrough of the scraping logic, challenge handling, and output structure is briefly explained below.

➤ Technology and Approach

The data was collected using a custom-built Python scraper combining Selenium and BeautifulSoup. Selenium was required because Stooq renders its data tables using JavaScript, meaning a standard HTTP request cannot retrieve the table content a real browser session is needed to fully load the page before parsing.

BeautifulSoup was then used to parse the rendered HTML and extract the table data row by row. The scraper navigated through all pages of each view using Stooq's pagination system, collecting data till the specified pages

➤ Key Challenges faced

- **Col span Headers:** Stooq's table headers use merged cells (colspan), requiring careful index mapping to correctly assign values to the right columns
- **Hidden Span Elements:** Each percentage value on Stooq is displayed in both a visible element and a hidden tag inside the same cell so the scraper had to explicitly target only the visible element to avoid duplicate or concatenated values
- **Delay and limits:** Stooq uses a rate limit for every users which is more than our program will ever reach and if the limit was reached we have to modify the code to use a visible browser window and clear captcha to gather data which can only happen during the development phase during the run of a ready script and the scraper included randomized wait times and retry logic to respect the website limits and not aggressively scrape the site.
- **Multi-View Architecture:** Data is split across three different URL views (v=1, v=4, v=6) which each load separately each view was scraped independently and saved to its own CSV to keep the scraping logic simple and modular
- **Pagination:** Each page only contain 100 entry per view and we had 3 views so in order to gather 6000 records for each view our scraped gathered 18000 rows. The page indexing was handled by URIs "l" parameter
- **Data:** A huge Number of Stocks did not have turnovers and Volume so we had to extensively filter them and reduce our dataset to approx. 4000 rows and fill remaining missing values

➤ Web Scraping Code Walkthrough

Configuration & Control Variables

The script is controlled by four variables at the top: `BASE_URL` and `INDEX_ID` point the scraper at Stooq's U.S. stock index (ID 518) Which is the page index for U.S Stocks Listing, `VIEWS` is a list that determines which data views to scrape (view 1 for prices, view 4 for returns, view 6 for 52-week metrics), and `PAGES` sets a Limit of 60 pages per view as a limit for No. of rows we need (60*100).

HTML Extraction & Hidden Element Removal (`parse_table()` Function)

This function receives the raw page HTML and returns the header row and data rows separately. It targets Stooq's data table using its fixed 'ID' `fth1`, which is consistent across all views and pages. Before extracting any text, it calls `decompose()` on all elements carrying `style="display:none"` this removes hidden `<span>` tags This issue was identified during the test run of script during development phase as Stooq embeds a hidden `<span>` tag inside percentage cells.

Without this step, calling `get_text()` on those cells concatenates the visible formatted value and the hidden raw value into one string (e.g. `+3.45%3.45`).

After cleaning, all `<tr>` rows are iterated and cell text is extracted from both `<th>` and `<td>` elements, returning the first row as headers and the rest as data.

Browser Setup & delay

Selenium launches Chrome in `"--headless=new"` mode so it runs invisibly. Before navigating to any data page, the scraper first loads Stooq's homepage and waits 3 seconds via `time.sleep(3)`. This is intentional so any loading can complete on first visit that must complete before data pages become accessible. Skipping this step causes the data table to never load.

Pagination Loop

For each page, the URL is constructed by injecting `INDEX_ID`, `v` (view), and `l` (page number) as query parameters. For delay, `WebDriverWait` with a 15-second timeout blocks execution until `fth1` actually appears in the DOM which give enough time for data to be actually loaded and become visible. Once loaded, `parse_table()` is called on `driver.page_source`. The stopping condition is when no data is available and fixed number of pages set is reached this reliably detect that all data has been collected.

DataFrame Assembly & Export

After all pages are collected each row is trimmed to `"len(headers)"` columns using slicing before building the Data Frame this solves the problem of extra columns caused by Colspan in the HTML. The file is exported and each view is saved as a separate CSV (`stooq_view1.csv`, `stooq_view4` etc.) keeping the outputs independent.

Data Cleaning & Preprocessing

Raw scraped data is rarely clean or analysis-ready. This section covers the full pipeline used to transform the three raw CSV files into a single, Cleaned, and complete dataset ready for EDA and ML modelling and evaluation.

➤ Merging The Files

During scraping the data we created 3 files (`stooq_view1.csv` ,`stooq_view4.csv` ,`stooq_view6.csv`).Where each file contains different featured of the stocks for example (`Turnover` , `Price` , `Returns` , `52Week change`...)

The three scraped CSV files were loaded independently and then merged into a single dataset using '`.merge()`' to perform a left join on the `Symbol` column. A left join was chosen so only the stocks present in all three files are stored in final dataset. The resulting merged dataset contains every stock from the daily price view, with return and 52-week metrics .

➤ Column Formatting & Type Conversion

Inspection of the raw merged dataset revealed that all return and change columns were stored as strings rather than numbers. This was caused by the formatting used on the Stooq website values like `+3.45%` are displayed with a plus sign prefix and a percentage suffix, which prevents automatic numeric conversion.

The cleaning process involved three steps: stripping the `%` suffix and `+` prefix from all 'return' and 'change' columns, renaming all percentage columns with a `_per` suffix (e.g. `1D` becomes `1D_return_per`) to preserve their meaning after removing the `%` sign, and then casting all cleaned columns to float.

The Volume and Turnover columns presented an additional challenge — Stooq uses shorthand notation where values are expressed as `1.5k` (thousands), `3.2m` (millions), or `1.1g` (billions). A custom `map_suffix()` function was built to detect the suffix, multiply by the corresponding multiple of 10, and return the raw numeric value.

➤ Missing Value Treatment

After initial inspection using `isnull().sum()`, the following issues were identified and resolved:

- **Empty column:** Completely empty column in the dataset were dropped entirely
- **Rows with null Volume AND Turnover:** Rows where both trading activity columns were missing carry no useful data removed to avoid noise without heavily reducing row count
- **Remaining null Turnover values:** Estimated using the formula: $\text{Turnover} = \text{Volume} \times \text{Average Price}$ (mean of Open, High, Low, Last) which is a standard financial approximation
- **All remaining nulls:** Filled with 0, indicating no data was available for that return period which is common for many stocks.

Finding: After cleaning, the dataset retained approximately 4,000+ valid rows with all columns fully populated and correctly typed and ready for segmentation and feature engineering.

➤ Segmentation and Feature Engineering

Before training any model, six rule-based segmentation columns were created to manually classify each stock across different dimensions.

These segmentations serve two purposes: they add categorical label to each stock, and they allow us to validate whether the model performed well or not.

All segmentations were applied using `pd.cut()` with explicitly defined bin boundaries and labels and those segmentations were as follows:

Liquidity Tier:

Stocks were classified by their daily trading Volume into five tiers. Volume is the most direct measure of how actively a stock is traded and is fundamental for understanding the market.

Tier	Daily Volume Range
Highly Liquid	> 1,000,000
Liquid	250,000 – 1,000,000
Moderate	50,000 – 250,000
Less Liquid	10,000 – 50,000
Very Less Liquid	< 10,000

Total Return Performance Trend:

For each stock, the number of return columns (out of 9, from 1D to YTD) that showed a positive value was counted. This score was then binned into a performance trend label. This segmentation captures consistency of performance across timeframes rather than any single return value.

Positive Periods (out of 9)	Label
0 – 1	Strong Negative Trend
2 – 3	Moderate Negative Trend
4 – 5	Mixed Trend
6 – 7	Moderate Positive Trend
8 – 9	Strong Positive Trend

52-Week Performance:

Stocks were segmented based on their `52W_Change_Per` column, which captures the total price change over the prior year. This allows us to identify extreme performers, steady growers, neutral stocks, and deeply declined stocks.

Label	52-Week Change Range
Crashed	< -50%
Underperformer	-50% to -17%
Neutral	-17% to +17%
Outperformer	+17% to +100%
High Flyer	> +100%

Short-Term & Long-Term Return Segments:

Two composite return segments were created by averaging return columns across short and long timeframes respectively. Short-term uses the average of the 1D, 3D, 5D, 10D, and 1M columns. Long-term uses the average of 2M, 3M, 6M, YTD, and 52W Change.

Short term		Long Term	
Strong Decline	< -10%	Strong Decline	< -30%
Moderate Decline	-10% to -2%	Moderate Decline	-30% to -10%
Stable	-2% to 2%	Stable	-10 to 10%
Moderate Growth	2% to 10%	Moderate Growth	10% to 50%
Strong Growth	> 10%	Strong Growth	> 50%

Price Tier

Stocks were classified by their last recorded closing price into five tiers. Price tier is a widely used dimension in financial analysis to distinguish between penny stocks, growth mid-caps, and established blue-chip names.

Tier	Last Closing Price
Penny Stock	< \$1
Low-Priced	\$1 – \$5
Mid-Priced	\$5 – \$50
High-Priced	\$50 – \$200
Blue Chip	> \$200

Feature Engineering Log Transformation:

Volume and **Turnover** were found to be extremely right-skewed, with skewness values of 48.62 and 17.37 respectively. This is caused by the massive difference in trading activity between micro stocks (which may trade only a few hundred shares per day) and large-cap stocks (which can trade hundreds of millions). Without correction, these two columns will reduce the accuracy of all the analysis and is especially dangerous during the training of the model. A **log1p()** transformation was applied to both columns to create **Volume_Log** and **Turnover_Log** to handle this issue.

Exploratory Data Analysis & Visualization

Before training the model, a series of visualizations were produced to understand the distribution of the data, validate the segmentations, and identify any patterns or anomalies that should be taken care of during modelling.

➤ Correlation Heatmap

A full correlation heatmap was generated across all numeric columns to check for strong linear relationships that might indicate redundant features or unexpected dependencies.

Finding: No strong correlations were found between price and volume features and return columns. Return columns showed the expected pattern of gradual correlation decay. 1D correlates strongly with 3D, moderately with 1M, and weakly with YTD.

➤ Liquidity Tier Distribution

A bar chart of the Liquidity Tier distribution revealed a heavily imbalanced picture the vast majority of stocks in the dataset fall into the Very Less Liquid tier, meaning they trade fewer than 10,000 shares per day.

Finding: This is expected and reflects the true composition of the U.S. stock market. Large, actively traded names represent a very small fraction of all listed securities. The dataset is dominated by small-cap, micro-cap, and thinly traded stocks.

➤ Liquidity Tier vs Average Stock Price Distribution

A secondary chart comparing average price by liquidity tier showed an inverse relationship as stocks

Finding: Very Less Liquid tier have the highest average closing price, while Highly Liquid stocks tend to be lower priced. This is consistent with the behavior of high-priced stocks that attract less retail volume

➤ Total Return Performance Trend Distribution

The performance trend distribution showed a strong skew toward the negative end. The Strong Negative Trend category contained the most stocks, meaning the majority of stocks in this sample had negative returns across most timeframes.

Finding: This is a common characteristic of broad market datasets. At any given point in time, the majority of listed stocks underperform.

➤ Short-Term Returns & Price Tier

A side-by-side chart compared the short-term return segment distribution and price tier distribution across the dataset.

Finding: Short-term returns were heavily concentrated in the Stable segment approximately 2,185 stocks showed minimal short-term price movement. This is consistent with the liquidity findings: thinly traded and dormant stocks rarely move much on a day-to-day or week-to-week basis. Price tiers were dominated by Mid-Priced (\$5–\$50) and Penny Stock (< \$1) categories, with Blue Chip stocks making up only a small fraction.

Model Building & Evaluation

➤ Model Selection

Given that the dataset has no target variable or labelled output, and all columns were mostly numerical, highly skewed, and has no time-series structure an unsupervised clustering approach was the most appropriate choice.

The Problem it solves of Filtering and Screening - Clustering enables the automatic segmentation of stocks into meaningful categories such as high-performing stocks, penny stocks, growth stocks, and blue-chip stocks. These insights can be delivered to stakeholders on a regular basis without requiring continuous manual analysis, thereby improving efficiency and scalability. The segmented groups can also support personalized stock recommendations by aligning stock characteristics with investor preferences and behavior. Furthermore, the framework can be enhanced by creating more better segments through the combination of multiple characteristics, such as **"High-Performing Penny Stocks"** or **"Underperforming Blue-Chip Stocks,"** allowing for deeper market insights and more targeted decision-making.

KMeans was selected as the algorithm for clustering the following reasons:

- **Data** : The data is entirely numeric, and contains no usable feature to derive a target regression or perform supervised classification, So clustering was the best approach
- **Dataset Size** : With approximately 4,500 rows and ~18 features, the dataset is perfectly Compatible with KMeans
- **Segmentation** : Segmentation that was done during preprocessing can only be best observed during clustering or classification
- **Interpretability** : KMeans is interpretable cluster centres can be directly examined to understand the average profile of each group

➤ Training Data Preparation

Before training, the dataset was filtered to remove all non-numeric columns, segmentation labels and log columns. This ensures the model learns purely from the raw price and return data without learning from manually created categories.

The training dataset retained the following feature groups: daily price columns (Open, High, Low, Last), trading Volume, Turnover, all the return columns (1D ,2D....., YTD), and the 52-week metrics (52W Change, 1Y High).

➤ Preprocessing Pipeline

A two-stage preprocessing pipeline was built using sklearn's Pipeline and ColumnTransformer to handle the scaling requirements of features:

- **Volume & Turnover**: $\log_{10}()$ followed by StandardScaler was used as these columns are extremely right-skewed (48 and 17) and all positive, making log transformation safe and effective
- **All Return, Price & Change Columns**: these columns are skewed but can contain negative values, making log transformation unsafe, So a StandardScaler() for these columns

➤ Optimal Cluster Selection — Elbow Method

To determine the optimal number of clusters, the Elbow Method was applied by running KMeans with Cluster values ranging from 2 to 11 and plotting the resulting for each value.

Finding: The inertia curve showed a clear change in slope (elbow) between $k=5$ and $k=6$, after which additional clusters produced reduced cluster quality. Based on this, $k=6$ was selected as the optimal cluster count, which also aligns well with manual segmentation rubrics.

➤ Final Model Configuration

The final clustering model was built using **K-Means** with **6 clusters**, as this number provided a good balance between cluster separation and interpretability based on the Elbow Method and business segmentation requirements.

The parameter **n_init=500** was used to run the algorithm with 500 different random centroid initializations, ensuring that the solution with the lowest within-cluster variance was selected and reducing the risk of converging to a suboptimal local minimum.

A fixed **random_state=32** was specified to ensure reproducibility, allowing the same clustering results to be generated across multiple runs. This configuration produced stable, well-defined clusters that effectively captured the diversity of stock characteristics within the dataset.

Three industry-standard metrics were used to evaluate the quality of the final KMeans clustering. Each metric measures a different aspect of cluster structure, and together they provide a complete and balanced picture of model performance.

➤ Evaluation

Three industry-standard metrics were used to evaluate the quality of the final KMeans clustering. Each metric measures a different aspect of cluster structure, and together they provide a complete and balanced picture of model performance

- **Silhouette Score: 0.2749** Moderate but meaningful cluster structure. Scores above 0.2 confirm real groupings exist in the data. Overlap is expected in financial data where stocks exist on a continuous spectrum.
- **Davies-Bouldin Score: 0.7263** Good separation. A score below 1.0 indicates that clusters are reasonably compact relative to their inter-cluster distances. Our score of 0.73 confirms well-defined groups.
- **Calinski-Harabasz: 1228.02** Strong density. Higher values indicate denser, more separated clusters. A score of 1228 on a 4,500-row dataset is a solid result confirming real structure was found.
- **Heat map :** After carefully examining all the cluster we found out that the clustering happen really well and clusters consisted of just one category from each segmentation in majority.

Finding: The evaluation confirms that KMeans successfully identified meaningful structure in the U.S. stock universe. Overall, the evaluation confirms the clustering is working well. The moderate Silhouette score is not a issue as it reflects the real-world stock market data, where no hard boundaries exist between groups. The Davies-Bouldin score below 1.0 and the high Calinski-Harabasz score together confirm that the clusters are genuinely compact and well-separated.

➤ Modeling Conclusion

The final model is a KMeans pipeline with k=6 clusters, using log+StandardScaler for Volume and Turnover and all other columns for all return and price. The model was evaluated using Silhouette Score metric and Visual Heatmap. The clustering successfully partitions the U.S. stock into meaningful and stable groups based on their price, volume, and return characteristics.

Out of 6 clusters, 3 cluster gave us very rare anomaly which were cluster – 1,2 and 5 and remaining cluster gave us stocks with different patterns that we were not able to segment manually

After carefully examining all the cluster we found out that the clustering happen really well and clusters consisted of just one category from each segmentation in majority. Further we labeled each cluster according to the clustering as follow:

cluster labels:

- **0: Dormant Small-Cap Stocks**
- **1: Ultra High-Price Outlier**
- **2: Extreme Historic Gainer Outlier**
- **3: High Momentum Growth Stocks**
- **4: Active Mid-Cap Mainstream Stocks**
- **5: Sub-Penny Anomaly**